

```
print("Computer Programming 2026\n")  
print("Department of Geography\n")  
print("19 March 2026\n")  
  
print("Week 4")
```

Exercise

Problem

- Find the largest number among N positive numbers that are typed on keyboard

Solution

- Know N
- Ask for inserting the N values
- Search for the largest among the N values

Exercise

```
N = int(input("How many numbers do you want to insert? "))

biggest = 0
count = 0
while count < N:
    num = int(input("Insert the new number (positive): "))
    if num > biggest:
        biggest = num
    count += 1

print("\nThe biggest number is", biggest)
```

Exercise

Problem

- There are 10 judges
 - 1 judge = 1 vote
- Each vote must fall within the interval 0-10
- Given a dive, calculate:
 - the average of the votes
 - the highest vote and the lowest vote

Exercise

```
judge = 1
high = 0
low = 0
tot = 0

while judge <= 10:
    vote = int(input(f"Insert the vote of judge {judge}: "))
    if vote > high:
        high = vote
    if vote < low:
        low = vote
    tot = tot + vote

print("\nThe highest vote is", high)
print("\nThe lowest vote is", low)
print("\nThe average is", tot / 10)
```



What happens to the variable "judge" at every iteration?

NOTHING!!!!
Infinite iterations!!!

Exercise

```
judge = 1
high = 0
low = 10
tot = 0

while judge <= 10:
    vote = int(input(f"Insert the vote of judge {judge}: "))
    if vote > high:
        high = vote
    if vote < low:
        low = vote
    tot = tot + vote
    judge += 1

print("\nThe highest vote is", high)
print("\nThe lowest vote is", low)
print("\nThe average is", tot / 10)
```

ANOTHER EXAMPLE

The factorial

Given n , positive integer, we define the n factorial (represented as $n!$) the product of the first n positive integer numbers that are lower or equal to that number.

$$n! := \prod_{k=1}^n k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

- Note:
 - $0! = 1$
 - $1! = 1$
 - $2! = 2, 3! = 6, \dots$

The factorial: code

```
factorial = 1

n = int(input("Of which number do you want to calculate the factorial? "))

i = n
while i > 1:
    factorial = factorial * i
    i -= 1

print(f"\nThe factorial of {n} is {factorial}")
```

PROBLEM!!!

WHAT IF $N = -5$?

SOLUTION!

ASK FOR N AGAIN IF < 0

The factorial: code

```
factorial = 1

while True:
    n = int(input("Of which number do you want to calculate the factorial? "))
    if n >= 0:
        break

i = n
while i > 1:
    factorial = factorial * i
    i -= 1

print(f"\nThe factorial of {n} is {factorial}")
```

```
while not edge:  
    run
```

```
while True:  
    run  
    if edge:  
        break
```



PROBLEM!!!

WHAT IF $N = 3.4$?

WHAT IF $N = \text{DOG}$?

SOLUTION!

**PREVENT THE PROGRAM
FROM CRASHING BY
RAISING AN EXCEPTION**

Try-Except statement

The idea of the try-except statement is to handle exceptions (errors at runtime)

```
try:  
    <do something>  
except:  
    <handle the error>
```

- **try:**
the code with the exception(s) to catch. If an exception is raised, it jumps straight into the except block.
- **except:**
this code is only executed *if an exception occurred* in the try block. The except block is required with a try block, even if it contains only the pass statement.

The factorial: code

```
factorial = 1

while True:
    try:
        n = int(input("Of which number do you want to calculate the factorial? "))
        if n >= 0:
            break
    except:
        print("The inserted value is not an integer number, please try again.")

i = n
while i > 1:
    factorial = factorial * i
    i -= 1

print(f"\nThe factorial of {n} is {factorial}")
```

The factorial: code

```
factorial = 1

while True:
    try:
        n = int(input("Of which number do you want to calculate the factorial? "))
        if n >= 0:
            break
        else:
            print("You inserted a negative number, please try again.")
    except:
        print("The inserted value is not an integer number, please try again.")

i = n
while i > 1:
    factorial = factorial * i
    i -= 1

print(f"\nThe factorial of {n} is {factorial}")
```


The factorial: code

```
factorial = 1

while True:
    try:
        n = int(input("Of which number do you want to calculate the factorial? "))
        if n >= 0:
            break
        else:
            print("You inserted a negative number, please try again.")
    except:
        print("The inserted value is not an integer number, please try again.")

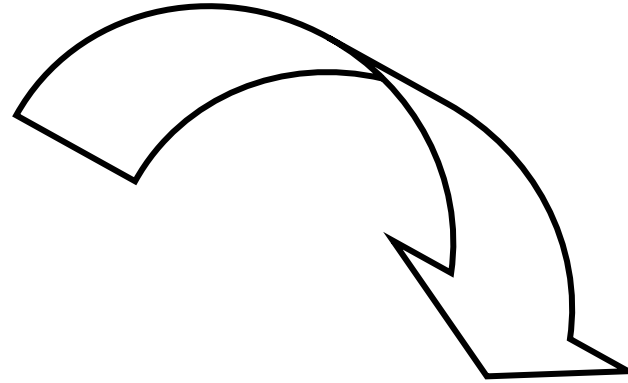
    i = n
    while i > 1:
        factorial = factorial * i
        i -= 1

print(f"\nThe factorial of {n} is {factorial}")
```

Observation: Why using a *while* if I **know** the number of iterations of the loop?

for loop

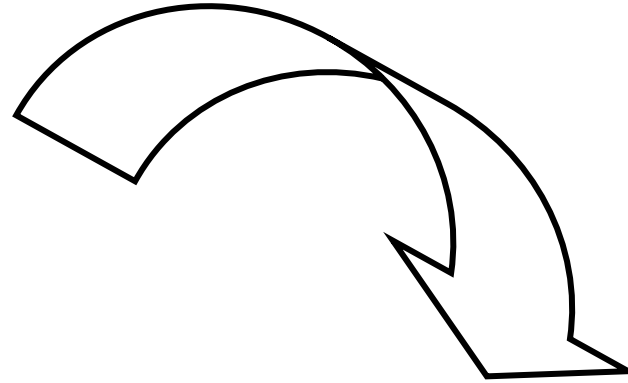
```
i = n  
while i > 1:  
    [...operations...]  
    i -= 1
```



```
for i in range(n, 1, -1):  
    [...operations...]
```

for loop

```
i = 0  
while i < N:  
    [...operations...]  
    i += 1
```



```
for i in range(0, N, 1):  
    [...operations...]
```

The factorial – while VS for

```
i = n
while i > 1:
    factorial = factorial * i
    i -= 1
```

VS

```
for i in range(n, 1, -1):
    factorial = factorial * i
```

The factorial – *for* : code

```
factorial = 1

while True:
    try:
        n = int(input("Of which number do you want to calculate the factorial? "))
        if n >= 0:
            break
        else:
            print("You inserted a negative number, please try again.")
    except:
        print("The inserted value is not an integer number, please try again.")

for i in range(n, 1, -1):
    factorial = factorial * i

print(f"\nThe factorial of {n} is {factorial}")
```


Exercise

Problem

- Find the biggest number among N positive numbers that are typed on keyboard

Solution

- Know N
- Ask for inserting the N values
- Search for the biggest among the N values

Exercise – while VS for

```
count = 0
while count < N:
    num = int(input("Insert the new number (positive): "))
    if num > biggest:
        biggest = num
    count += 1
```

VS

```
for count in range(N):
    num = int(input("Insert the new number (positive): "))
    if num > biggest:
        biggest = num
```

Exercise

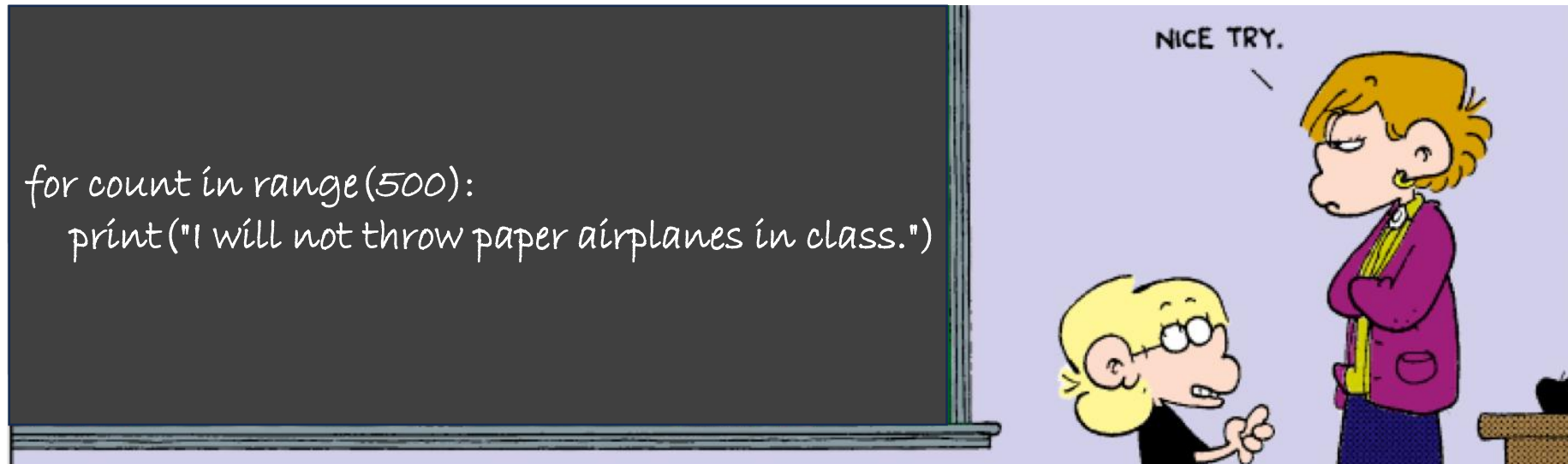
```
N = int(input("How many numbers do you want to insert? "))

biggest = 0

for count in range(N):
    num = int(input("Insert the new number (positive): "))
    if num > biggest:
        biggest = num

print("\nThe biggest number is", biggest)
```

Now it should be clear...



Objectives

- Functions
- Global vs local variables

The factorial

Given n , positive integer, we define the n factorial (represented as $n!$) the product of the first n positive integer numbers that are lower or equal to that number.

$$n! := \prod_{k=1}^n k = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

- Note:
 - $0! = 1$
 - $1! = 1$
 - $2! = 2, 3! = 6, \dots$

**FROM THE
FACTORIAL...**

**...TO THE BINOMIAL
COEFFICIENT!**

Binomial coefficient

The number of choices of **k** objects among the ones that form a set of **n** elements

(therefore, the number of the subsets of **k** elements of a given set of **n** objects)

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Binomial coefficient

f1=fact(n)

f2=fact(k)

f3=fact(n-k)

res=f1/(f2*f3)

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Binomial coefficient

f1 = fact(n)

f2 = fact(k)

f3 = fact(n-k)

res = **f1** / (f2 * f3)

$$\binom{n}{k} = \frac{\textcolor{red}{n}!}{k! \cdot (n-k)!}$$

Binomial coefficient

$f1 = \text{fact}(n)$

$f2 = \text{fact}(k)$

$f3 = \text{fact}(n-k)$

$\text{res} = f1 / (f2 * f3)$

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Binomial coefficient

$f1 = \text{fact}(n)$

$f2 = \text{fact}(k)$

$f3 = \text{fact}(n-k)$

$\text{res} = f1 / (f2 * f3)$

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Binomial coefficient: workflow

1. Insert K and N
2. Verify K and N
3. If correct
 - A. Calculate the factorial of N (FactN)
 - B. Calculate the factorial of K (FactK)
 - C. Calculate the factorial of N-K (FactNK)
 - D. $\text{BinCoef} = \text{FactN} / (\text{FactK} * \text{FactNK})$
4. Else, go back to 1

But...

What is «*Calculate the factorial of N* »?

What is «*Calculate the factorial of K* »?

What is «*Calculate the factorial of $N-K$* »?

They are all **SUBPROGRAMS!!**

Subprograms

- **A subprogram is:**
 - a set of instructions with a name
 - described (defined) only once
 - potentially activated (called) inside a program or another subprogram
- **Some subprograms are already defined**
 - think about the *input* and the *print*
 - behind these names there are groups of instructions that allow monitoring signals from keyboard and visualizing characters on screen, respectively
 - if you call these functions, you don't need to worry about how they are made, you only care of what they do (black-box view)

Subprograms: purpose

- **Abstraction and readability:**
 - summarize code, hiding algorithmic details
 - the name of the subprogram is presented as “an elementary operation”
- **Functional organization and decomposition of the program:**
 - allow for writing a program in a way that reflects a functional analysis of the problem
- **Testing:**
 - verifying the correctness of a solution is easier if you: first verify the correctness of the individual subprograms, and then the correctness of the whole program (seen as a set of subprograms that exchange information between them)
- **Code compactness and efficiency:**
 - avoid repeating the same sequences of instructions in different parts of the program
- **Adjustability:**
 - only one modification has effect on every activation of the subprogram
- **Re-use:**
 - subprograms that are not too specific can be grouped into libraries and used in different programs

Subprograms

Subprograms can be of two different types

- *Functional* type
 - *Procedural* type
- Functional-type subprograms can be considered as an abstraction of value
 - calling the function means to associate to the name of the function the value representing the calculated result of the subprogram

```
result = sum(A,B)
```

- Procedural-type subprograms can be considered as an abstraction of operations
 - calling the function means to execute the instructions of the subprogram, which perform the operation specified by the subprogram

```
print(A,B)
```

Functions

Using a function implies:

- 
1. Defining the function
 2. Calling the function (wherever the function is needed in the code)

1. Defining the function

- The function definition should be executed before you call such function in the main code

- The head of the function should be written as:

```
def name_of_the_function(par1, par2, ...):
```

- `par1, par2, ...` are the names of the formal parameters of the function

Return instruction

- Keyword used in the functions
- Syntax:

```
return some_kind_of_value
```

- It is the last instruction of a function and indicates:
 - the point when the control is back to the caller
 - the returned value
- In a function:
 - there must be at least one return instruction
 - there may be more than one return instructions, but alternatively
 - the function may also return several values

Example: factorial function

```
def fact(val):  
    factorial = 1  
  
    for i in range(val, 1, -1):  
        factorial = factorial * i  
  
    return factorial
```

Functions

Using a function implies:

1. Defining the function

-  2. Calling the function (wherever the function is needed in the code)

2. Calling the function

Within your program, it indicates the location where the code inside the function definition must be executed.

- Calling a function:

- as an operation implying a “visible” outcome:

```
outcome = functionname(parameter1, ...)
```

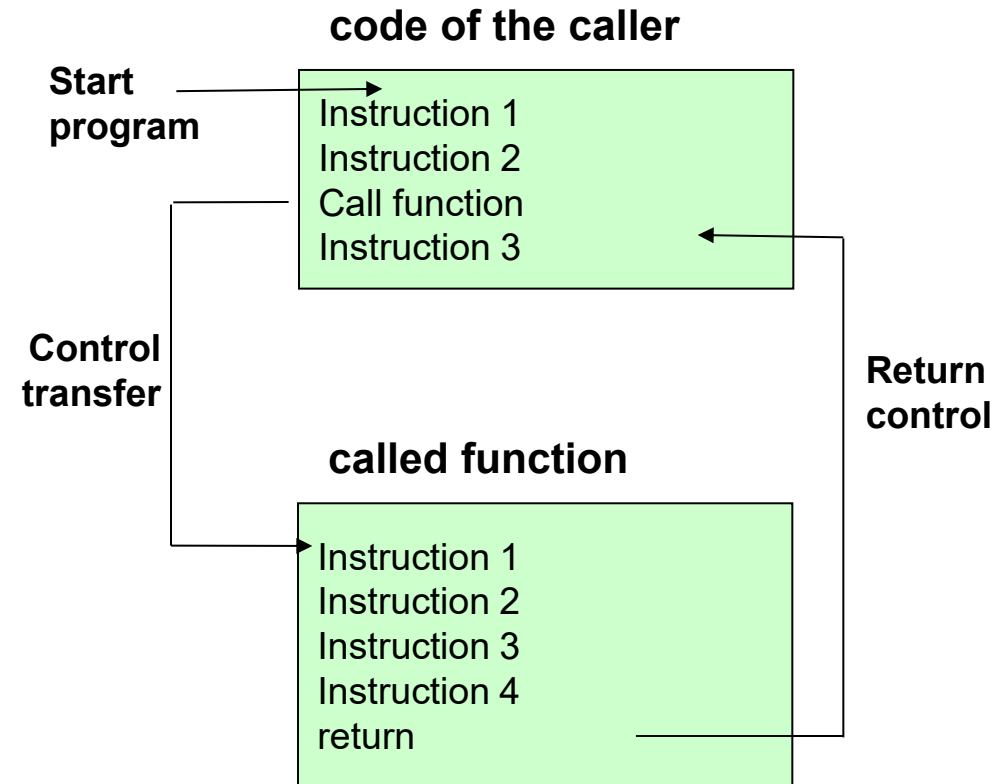
- as an instruction

```
functionname(parameter1, ...)
```

- `Parameter1, ...` are the parameter values that must match, in terms of order and type, the formal parameters of the function (as defined in the definition step)
- The “parameter values” can be variables, constants, or expressions, whose values are copied in the formal parameters of the function at the moment you call the function

Caller/Called

- At calling time, the control of the execution moves from the caller to the called
- The code of the called is executed
- At the end of the execution, the control is back to the caller, to the instruction that follows the call



Back to the binomial coefficient

1. Insert K and N
2. Verify K and N
3. If correct
 - A. Calculate the factorial of N (FactN)
 - B. Calculate the factorial of K (FactK)
 - C. Calculate the factorial of N-K (FactNK)
 - D. $\text{BinCoef} = \text{FactN} / (\text{FactK} * \text{FactNK})$
4. Else, go back to 1



Binomial coefficient

Input constraints:

$$0 \leq k \leq n$$

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Conditional statement

- **P1**: $K \geq 0$
- **P2**: $N \geq K$

P1	P2
0	0
0	1
1	0
1	1

Conditional statement

- **P1**: $K \geq 0$
- **P2**: $N \geq K$

P1	P2	
0	0	0
0	1	
1	0	
1	1	

Conditional statement

- **P1**: $K \geq 0$
- **P2**: $N \geq K$

P1	P2	
0	0	0
0	1	0
1	0	
1	1	

Conditional statement

- **P1**: $K \geq 0$
- **P2**: $N \geq K$

P1	P2	
0	0	0
0	1	0
1	0	0
1	1	

Conditional statement

- **P1**: $K \geq 0$
- **P2**: $N \geq K$

P1	P2
0	0
0	1
1	0
1	1

0

0

0

1

DO YOU

REMEMBER?

Logical fundamental operations

<u>A</u>	<u>B</u>	<u>A or B</u>
0	0	0
0	1	1
1	0	1
1	1	1

(logical sum)

<u>A</u>	<u>B</u>	<u>A and B</u>
0	0	0
0	1	0
1	0	0
1	1	1

(logical product)

<u>A</u>	<u>not A</u>
0	1
1	0

(negation)

The tables list all possible input combinations and the corresponding result of each combination

Conditional statement

- $P1: K \geq 0$
- $P2: N \geq K$

<u>P1</u>	<u>P2</u>	<u>P1 AND P2</u>
0	0	0
0	1	0
1	0	0
1	1	1

Binomial coefficient: code

```
while True:
    K = int(input("\nInsert the value of K: "))
    N = int(input("Insert the value of N: "))
    if K >= 0 and N >= K:
        break
```

Back to the binomial coefficient

1. Insert K and N
2. Verify K and N
3. If correct
 - A. Calculate the factorial of N (FactN)
 - B. Calculate the factorial of K (FactK)
 - C. Calculate the factorial of N-K (FactNK)
 - D. $\text{BinCoef} = \text{FactN} / (\text{FactK} * \text{FactNK})$
4. Else, go back to 1



Binomial coefficient

$$f1 = \text{fact}(n)$$

$$f2 = \text{fact}(k)$$

$$f3 = \text{fact}(n-k)$$

$$\text{ris} = f1 / (f2 * f3)$$

$$\binom{n}{k} = \frac{n!}{k! \cdot (n-k)!}$$

Binomial coefficient: code

```
FactK = fact(K)
FactN = fact(N)
FactNK = fact(N-K)

BinCoef = FactN / (FactK * FactNK)

print(f"\nThe Binomial Coefficient with N = {N} and K = {K} is {int(BinCoef)}")
```

```
def fact(val):  
    factorial = 1  
    for i in range(val, 1, -1):  
        factorial = factorial * i  
    return factorial
```

```
while True:  
    K = int(input("\nInsert the value of K: "))  
    N = int(input("Insert the value of N: "))  
    if K >= 0 and N >= K:  
        break
```

```
FactK = fact(K)  
FactN = fact(N)  
FactNK = fact(N-K)  
BinCoef = FactN / (FactK * FactNK)  
print(f"\nThe Binomial Coefficient with N = {N} and K = {K} is {int(BinCoef)}")
```

Parameter transfer

The parameter transfer consists of associating, at the moment of calling a subprogram, the current parameters to the formal parameters

- If a function is

```
def fact(val) :
```

- Calling this function means to execute the instruction

```
FactK = fact(K)
```

- In this way, the formal parameter will assume, in that specific call, the value of the current parameter (e.g., the value 5).

Formal parameters

- They represent a symbolic reference (identifiers) to objects used inside the function
- They are used by the function as if they were local variables
- The initial value of the formal parameters is defined at the moment of calling the function, based on the current parameters (parameter transfer)

```
def fact(val):  
    factorial = 1  
  
    for i in range(val, 1, -1):  
        factorial = factorial * i  
  
    return factorial
```

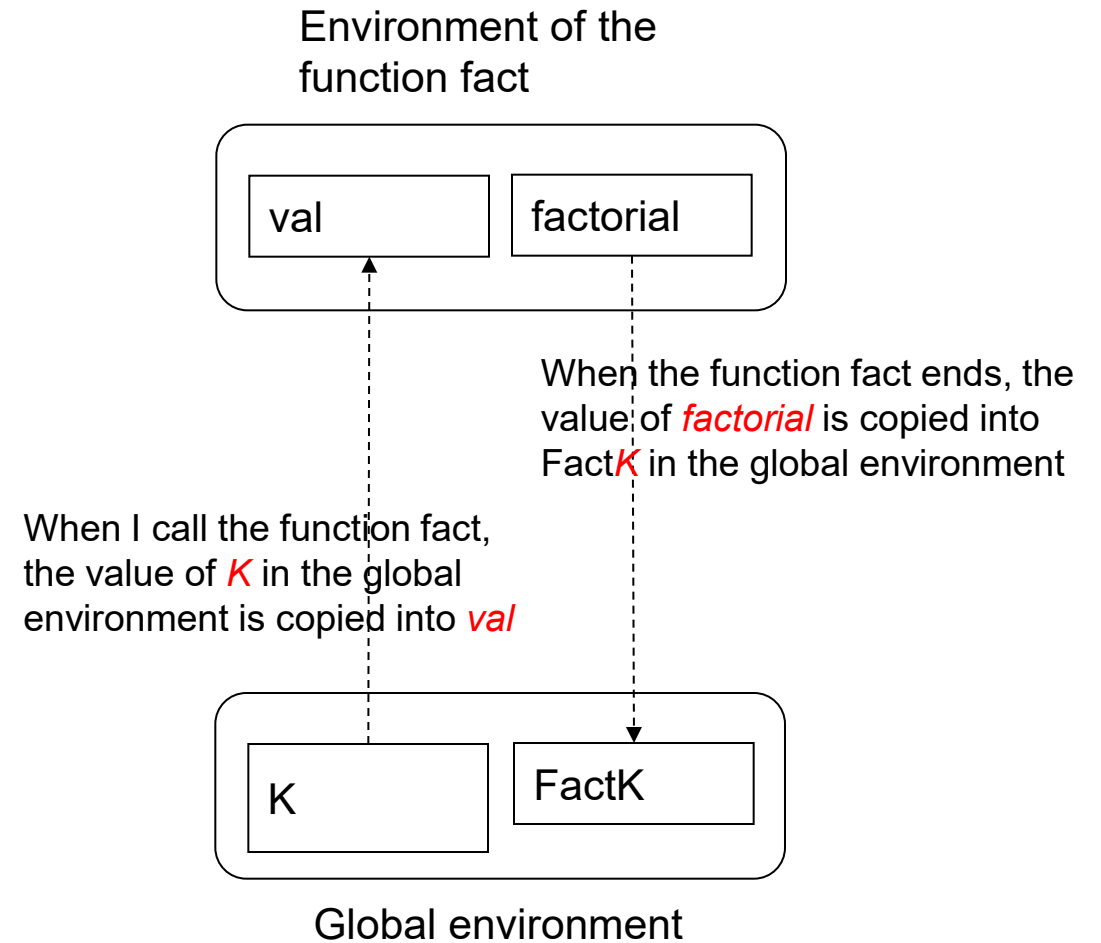
Parameter transfer

- At the time of calling the function, the value of the current parameter is copied into the corresponding formal parameter.
 - The formal parameter and the current parameter **refer to two different memory cells**
- The subprogram, during execution, works in its own environment, therefore on the formal parameters
- The current parameters are not modified

Example

```
def fact(val):  
    factorial = 1  
  
    for i in range(val, 1, -1):  
        factorial = factorial * i  
  
    return factorial
```

FactK = fact(K)



Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.

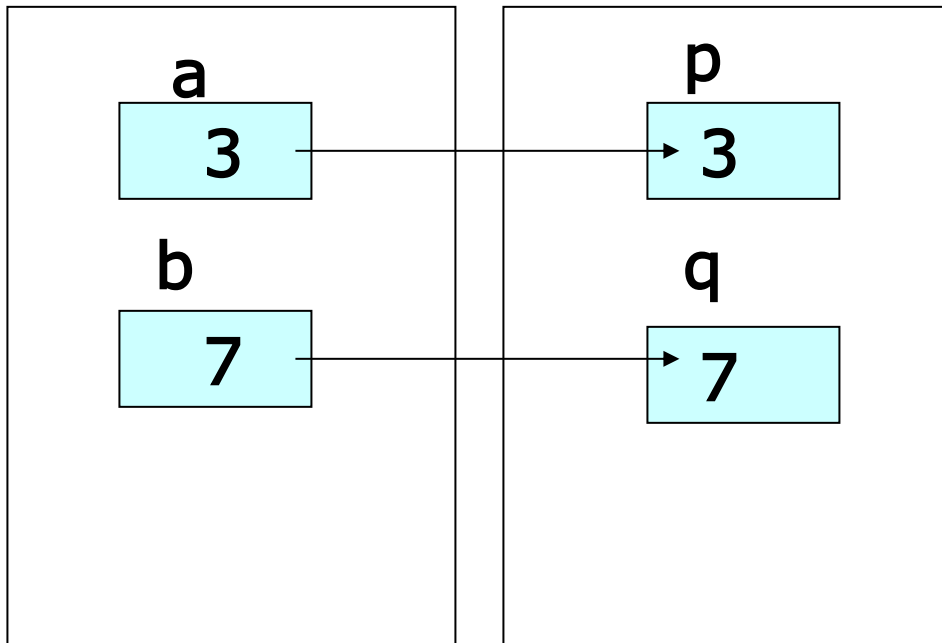
```
...  
temp = p  
p = q  
q = temp  
...
```

```
def swap(p,q):  
    temp = p  
    p = q  
    q = temp  
    return p,q
```

```
a,b = swap(a,b)
```

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.



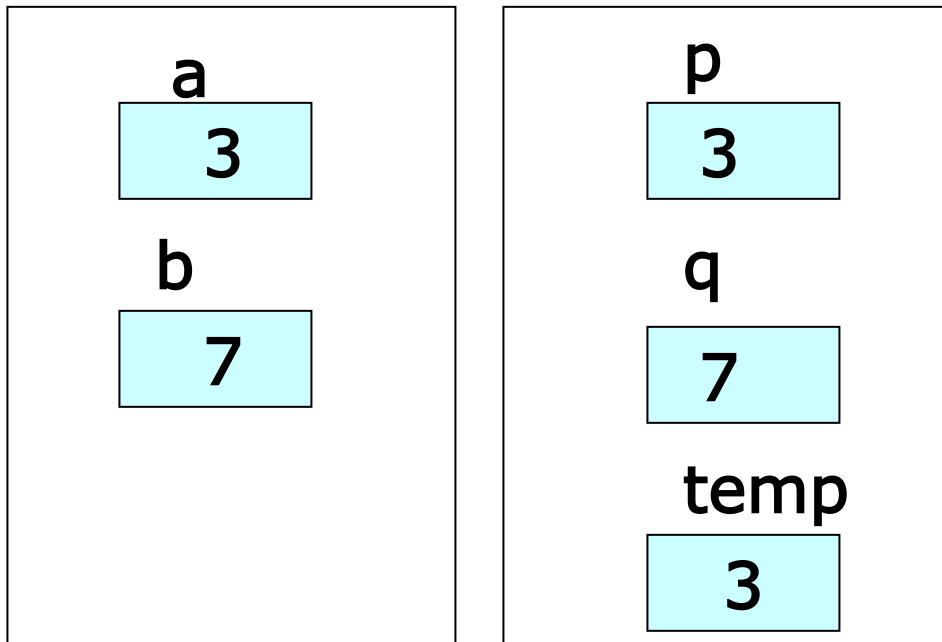
→

```
def swap(p,q):  
    temp = p  
    p = q  
    q = temp  
    return p,q
```

`a,b = swap(a,b)`

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.

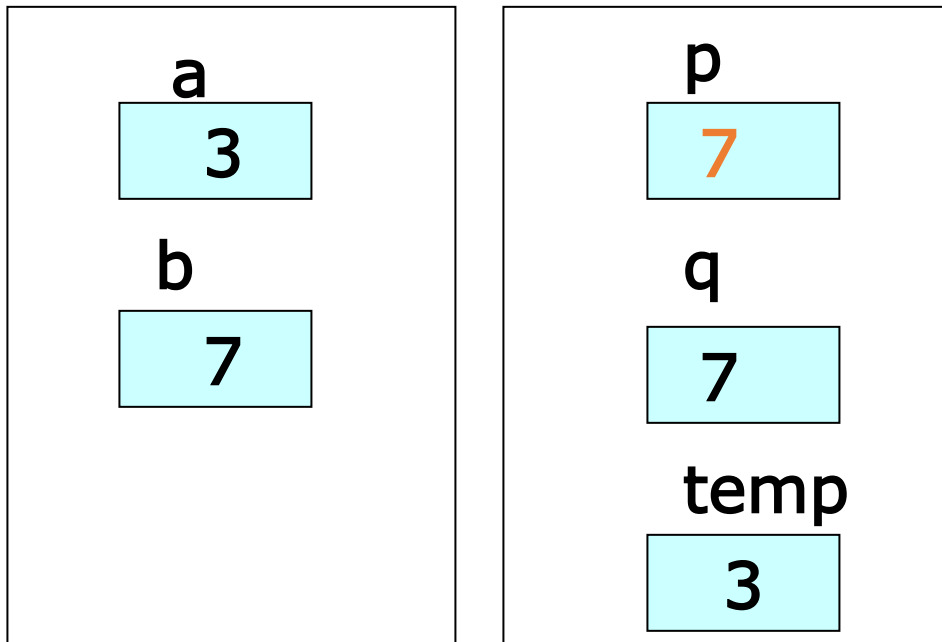


```
def swap(p,q):  
    → temp = p  
      p = q  
      q = temp  
      return p,q
```

```
a,b = swap(a,b)
```

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.

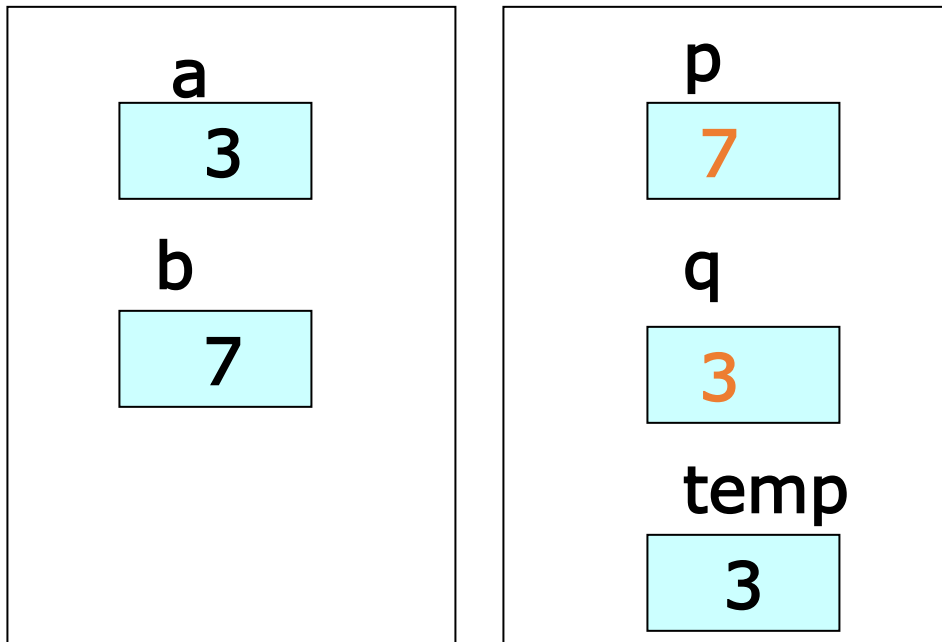


```
def swap(p,q):  
    temp = p  
    p = q  
    q = temp  
    return p,q
```

`a,b = swap(a,b)`

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.

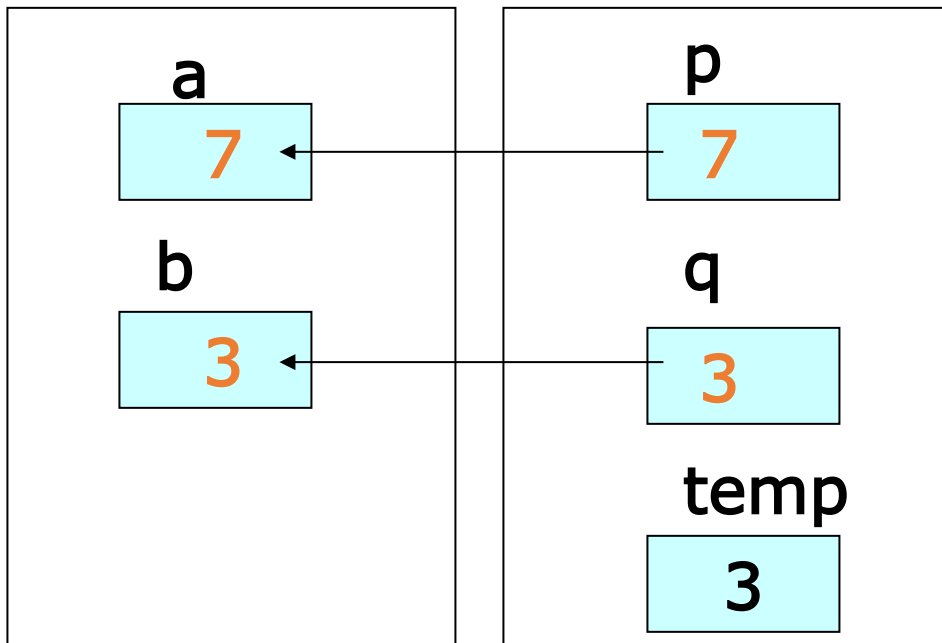


```
def swap(p,q):  
    temp = p  
    p = q  
    q = temp  
    return p,q
```

→
`a,b = swap(a,b)`

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.



```
def swap(p,q):  
    temp = p  
    p = q  
    q = temp  
    return p,q
```

→
`a,b = swap(a,b)`

VISIBILITY *EFFECT*!



Visibility

- **Visibility of an identifier:**
 - Indication of the part of the program where such identifier can be used
- **Global environment of the program**
 - set of identifiers (variables, constants, ...) in the global environment
 - visibility rules: visible to all functions in the program
- **Local environment of a function**
 - set of identifiers in the local environment and formal parameters
 - visibility rules: visible to the function only

Variables' visibility

```
result = 6
```

```
a = addition(2,1)
```

```
b = subtraction(4,1)
```

```
def addition(d1,d2):  
    result = d1 + d2  
    return result
```

```
def subtraction(d1,d2):  
    result = d1 - d2  
    return result
```

Shadowing

- A name redefined inside a local environment hides the previous meaning of that name
- Such meaning is reinstated at the end of the local environment

```
result = 6
```

```
a = addition(2,1)
```

```
def addition(d1,d2):  
    result = d1 + d2  
    return result
```

- In case of identifiers with the same name in different environments, only the one in the “closest” environment is visible.

Shadowing

```
result = 6
```

```
addition(2,1)
```

6

```
def addition(d1,d2):  
    result = d1 + d2  
    return
```

```
result = [6]
```

```
addition(2,1)
```

[3]

```
def addition(d1,d2):  
    result[0] = d1 + d2  
    return
```

LIST

Variable vs List

```
def add_one(x):  
    x += 1  
    return
```

```
y = 23
```

```
add_one(y)
```

y

23

```
def add_one(x):  
    x[0] += 1  
    return
```

```
y = [23]
```

```
add_one(y)
```

y

[24]

Variable vs List

```
def add_one(x):  
    x += 1  
    return
```

```
y = 23
```

```
add_one(y)
```

y

23

```
def add_one(x):  
    x[0] += 1  
    return
```

```
y = [23]
```

```
add_one(y)
```

y[0]

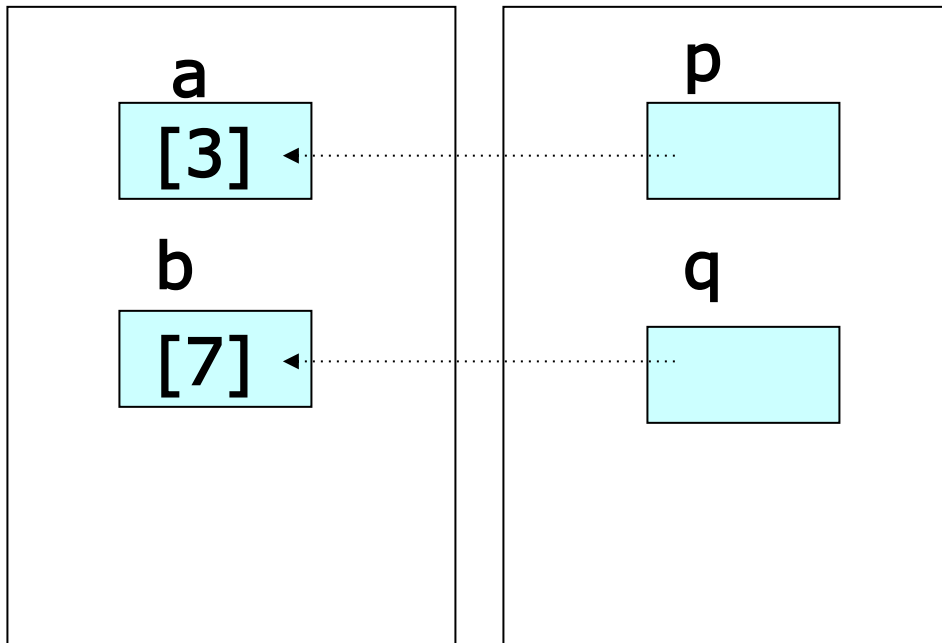
24

Parameter transfer (list)

- At calling time, the current parameters are linked to the formal parameters
 - the current parameter and the formal parameter refer to the same **memory cell**
- The subprogram, during execution, works in its environment on the formal parameters (and consequently also on the current parameters)
 - every modification on the formal parameter is a modification of the corresponding current parameter
- The effects of the subprogram arises in the caller, with modifications in the caller's environment

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.



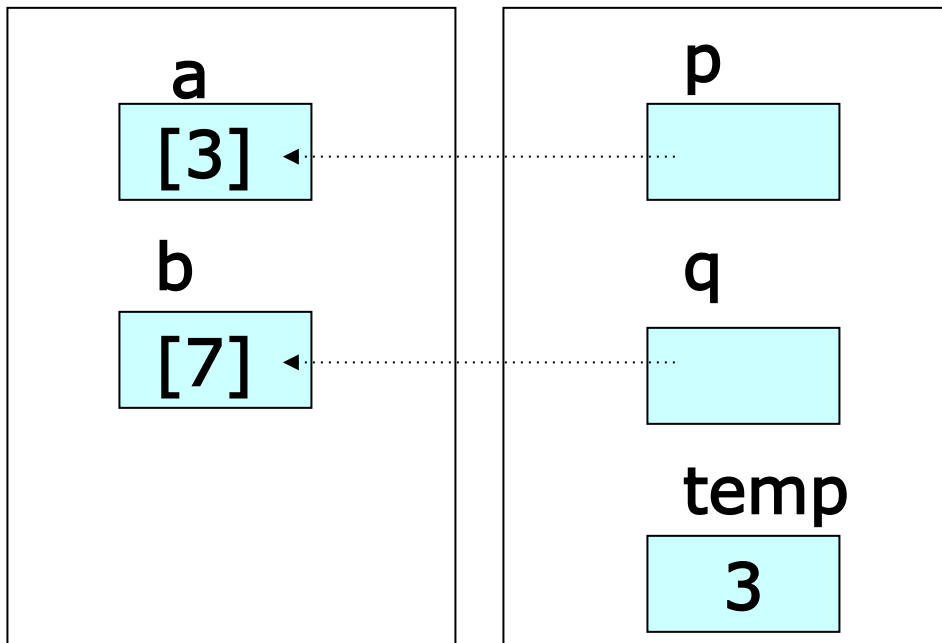
➡

```
def swap(p,q):  
    temp = p[0]  
    p[0] = q[0]  
    q[0] = temp  
    return
```

`swap(a,b)`

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.

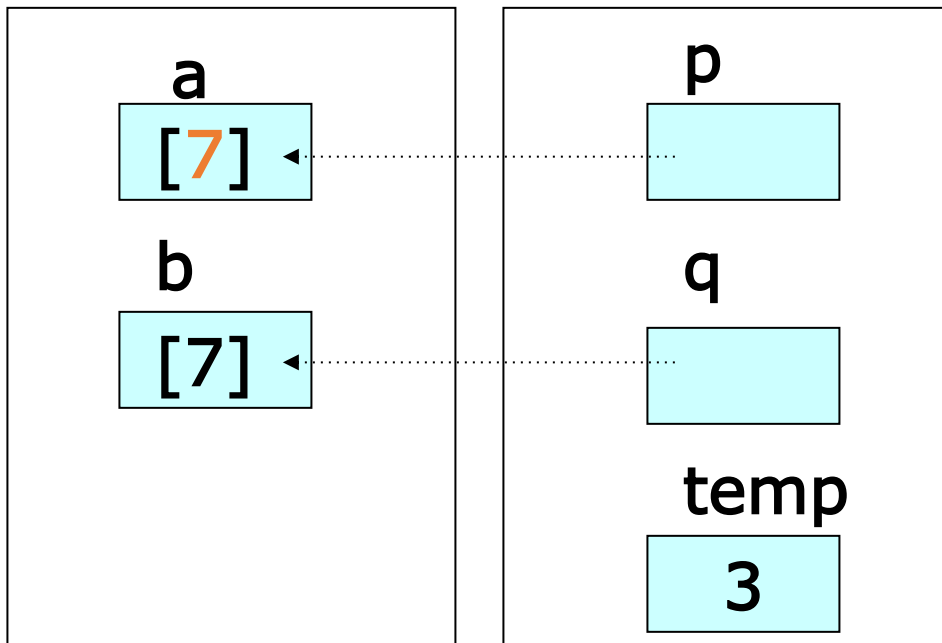


```
def swap(p,q):  
    temp = p[0]  
    p[0] = q[0]  
    q[0] = temp  
    return
```

swap(a,b)

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.

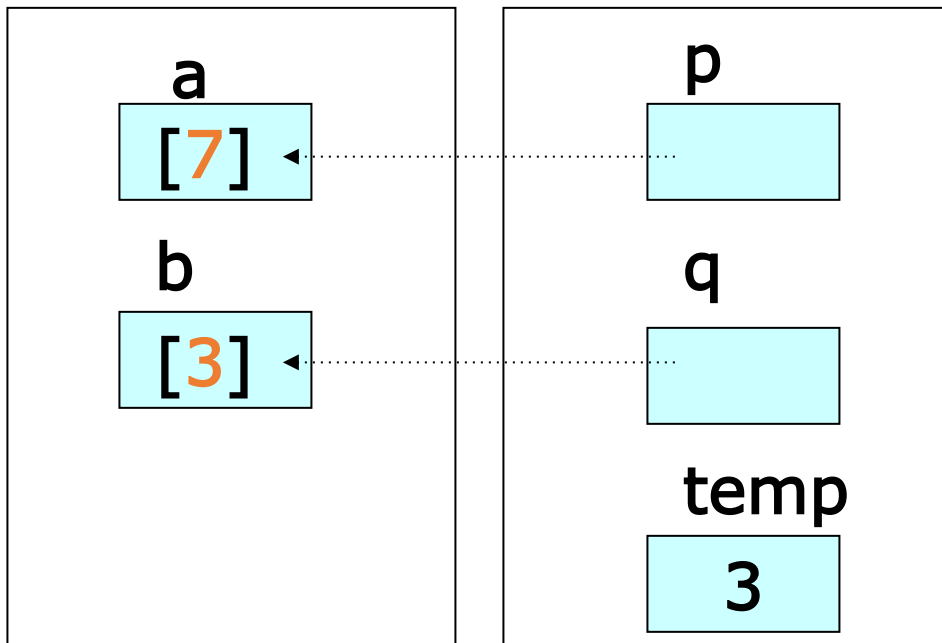


```
def swap(p,q):  
    temp = p[0]  
    p[0] = q[0]  
    q[0] = temp  
    return
```

swap(a,b)

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.



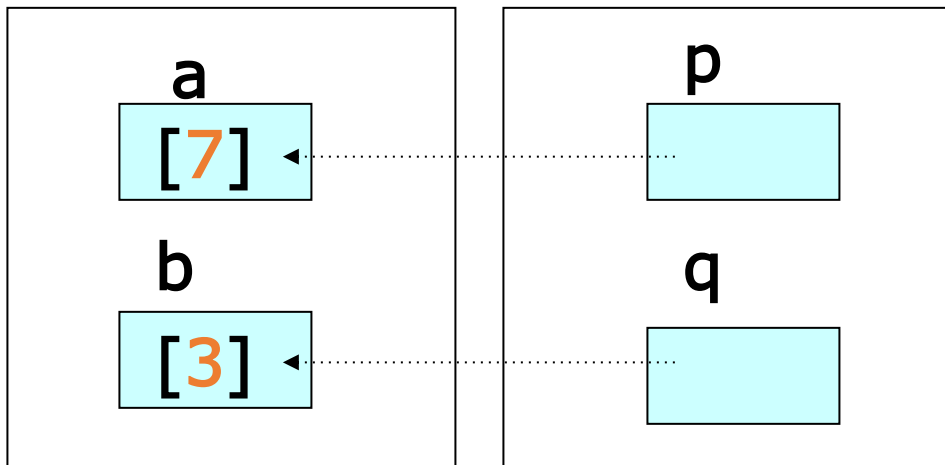
→

```
def swap(p,q):  
    temp = p[0]  
    p[0] = q[0]  
    q[0] = temp  
    return
```

`swap(a,b)`

Exercise

Given a variable p (integer value) and a variable q (integer value), write a program that switches the value of the two variables.



The values in the global environment are directly modified

```
def swap(p,q):  
    temp = p[0]  
    p[0] = q[0]  
    q[0] = temp  
    return
```

```
swap(a,b)
```



See you
next week!